Relazione Progetto Formula Uno

Introduzione

Il progetto sviluppato è un'applicazione di simulazione di una gara di Formula 1 (Vector Rally), realizzata in Java con l'ausilio di Gradle per la gestione dei build e delle dipendenze. L'applicativo è progettato per essere eseguito tramite console, fornendo una simulazione dettagliata e interattiva del gioco.

Di seguito, viene fornita una panoramica del funzionamento dell'applicativo e delle responsabilità delle principali classi.

Funzionamento dell’applicativo

All'avvio dell'applicativo, se non vengono forniti dati specifici, il programma utilizza impostazioni di default per la pista e il numero di giocatori. I giocatori sono trattati come bot, ma il sistema è progettato per poter essere esteso in futuro per supportare giocatori umani.
La simulazione avviene su una griglia, e i bot vengono posizionati casualmente sulla griglia di partenza.

La pista può essere personalizzata creando file di testo con coordinate, seguendo un formato specifico per garantire un parsing corretto e senza errori.
Durante il gioco, il percorso è definito da un singolo giro dalla partenza al traguardo, e la gara si svolge in turni, con ogni giocatore che sceglie la propria mossa in modo sequenziale.

Il vincitore è determinato da chi raggiunge per primo la sezione del traguardo. Alla fine della gara, il risultato viene visualizzato, mostrando le posizioni finali dei giocatori e il numero di mosse (tempo) impiegato.

Responsabilità delle classi

Responsabilità: Gestire la simulazione dei movimenti dei bot sulla pista di Formula 1.
(Determina la prossima mossa del bot, verifica se la velocità è adeguata e gestisce le condizioni di vittoria o collisione).
Implementata con:

```
ControllerFormulaUnoBotSimulator implements ControllerBotSimulator
```

Responsabilità: Coordinare l'inizio e il controllo della partita, quindi gestione della creazione della mappa, l'inizio del gioco e le informazioni di gioco.
Implementata con:

```
ControllerFormulaUnoEngine implements ControllerGameEngine
```

Responsabilità: Controllare le collisioni verificando se le posizioni sulla pista sono libere.
Implementata con:

```
FormulaUnoEmptyPositionChecker
```

Responsabilità: Verificare le regole del gioco e coordinare le mosse dei giocatori.
Implementata con:

```
FormulaUnoGameDirector
```

Responsabilità: Generare le informazioni di gioco da visualizzare.
Implementata con:

```
FormulaUnoInfoBuilder
```

Responsabilità: Creare la mappa a partire dal raggio e dalle coordinate fornite.
Implementata con:

```
FormulaUnoMapCreator
```

Responsabilità: Controllare la correttezza del formato della mappa dal file di testo con successivo parsing delle informazioni per costruire il tracciato custom.
Implementata con:
`FormulaUnoMapParser`

Responsabilità: Scegliere la strategia di gioco di un giocatore.
(Attualmente la sceglie casualmente, riducendo la possibilità, però, di estrarre troppo spesso le strategie più "PERICOLOSE" (*maxSpeed > 3*), così da rendere il
gioco un po' più intrattenitivo e piacevole.)
Implementata con:
`PlayerStrategySelector`

Responsabilità: Fornire coordinate bidimensionali (x,y) e metodi per operazioni geometriche.
Implementata con:
`Coordinate`

Responsabilità: Rappresentare un giocatore di Formula Uno.
(La classe astratta viene poi estesa dal giocatore controllato dal bot, *FormulaUnoBotPlayer*)
Implementata con:
`FormulaUnoPlayer`

Responsabilità: Rappresentare un giocatore di Formula Uno controllato dal bot (infatti vi è integrato il *ControllerFormulaUnoBotSimulator*).
Implementata con:
`FormulaUnoBotPlayer extends FormulaUnoPlayer`

Responsabilità: Rappresentare la mappa (il tracciato) del gioco Formula Uno.
Implementata con:
`FormulaUnoMap`

Responsabilità: Rappresentare una sezione della mappa del gioco Formula Uno.
(La mappa del gioco è composta da più sezioni).
Implementata con:
`FormulaUnoSection`

Responsabilità: Fornire una mappa di default, per effettuare una simulazione, in caso non ci sia un input da parte dell'utente.
Implementata con:
`FormulaUnoDefaultMap`

Responsabilità: Rappresentare una mossa nel gioco Formula Uno. Essa contiene tutte le informazioni necessarie: posizione precedente, posizione successiva, velocità precedente, velocità successiva e lo stato del giocatore dopo la mossa.
Implementata con:
`FormulaUnoMove`

Responsabilità: Leggere gli argomenti passati nella riga di comando per configurare l'applicativo.
`FormulaUnoViewArgsReader`

Responsabilità: Rappresentare il gioco sulla riga di comando, gestendo l'input e l'output della console.
`FormulaUnoViewController`

Struttura dell'applicativo

I dati custom vengono passati alla view (*FormulaUnoViewController*), vengono letti dal *FormulaUnoViewArgsReader* e successivamente viene istanziato un *ControllerFormulaUnoEngine*.

L'applicativo viene strutturato attorno a una classe principale chiamata *ControllerFormulaUnoEngine*, il motore di gioco, che inizializza la partita e procede in sequenza per effettuare una simulazione effettiva della gara, sfruttando le funzionalità delle altre classi, creando i giocatori e poi istanziando tutti gli altri oggetti utili.

Istanzia un parser (*FormulaUnoMapParser*) per leggere i dati del tracciato dal file indicato con il path ("-mp") durante l'avvio del jar e il numero dei giocatori ("-pn"), successivamente istanzia un creatore del tracciato (*FormulaUnoMapCreator*) passandogli i dati letti dal parser.

Dopo la creazione della pista si avvia il gioco, istanziando:

- un oggetto che si occupa di controllare le posizioni vuote nel gioco Formula Uno (*FormulaUnoEmptyPositionChecker*);
- un oggetto che si occupa della simulazione delle mosse dei bot (*ControllerFormulaUnoBotSimulator*);
  - All'interno del *FormulaUnoBotPlayer* viene istanziato un oggetto *PlayerStrategy* a cui viene assegnata una strategia casualmente, tramite il *PlayerStrategySelector*. Gli oggetti di tipo *PlayerStrategy* descrivono la velocità massima a cui possono andare tramite delle enumerazioni.
  - Sempre il GameEngine, poi li posiziona sulla griglia di partenza della pista, tramite l'oggetto *FormulaUnoSection*, che sfrutta la classe *Coordinate*.

Infine inizializza due oggetti:

- un arbitro che, prima di partire, casualmente sceglie l'ordine con il quale i giocatori partiranno, per essere coerente e imparziale a chi, eventualmente, è avvantaggiato dalla posizione in cui si trova rispetto alla prima sezione della pista. Poi controlla le mosse dei bot, decreta se hanno vinto o se sono andati fuori pista e/o avuto una collisione (*FormulaUnoGameDirector*);

- un costruttore di informazioni (*FormulaUnoInfoBuilder*) che fornisce man mano tutte le informazioni sullo svolgimento della gara.

ControllerFormulaUnoEngine

La classe ControllerFormulaUnoEngine gestisce l'inizio e il controllo di una partita di Formula 1.
Implementa l'interfaccia *ControllerGameEngine* e si occupa della creazione della mappa, dell'inizio del gioco e della gestione delle informazioni di gioco.

Metodo `startGame()`:
Funzionalità: Inizia una nuova partita creando i giocatori e il simulatore del bot. Aggiunge i giocatori alla mappa e inizializza il *FormulaUnoGameDirector* e *FormulaUnoInfoBuilder*.

Metodo `getInfo()`:
Funzionalità: Restituisce le informazioni correnti del gioco come un oggetto FormulaUnoInfo contenente le informazioni della partita.

Metodo `runRound()`:
Funzionalità: Esegue un turno di gioco, ottenendo la mossa del giocatore corrente e applicandola.

ControllerFormulaUnoBotSimulator

La classe ControllerFormulaUnoBotSimulator gestisce la simulazione dei movimenti di un bot all'interno di una pista di Formula 1. Implementa l'interfaccia *ControllerBotSimulator* e fornisce logica per determinare la prossima mossa del bot.

Nel costruttore, per ogni nuova istanza, viene iniettato un *FormulaUnoEmptyPositionChecker* (che gli serve per potersi muovere cercando di evitare il più possibile le collisioni, sapendo la sua posizione, la velocità e verificando dove siano gli altri giocatori) e gli viene passato il tracciato della gara.

Nel costruttore della classe *FormulaUnoBotPlayer*, tramite l'oggetto *PlayerStrategySelector*, gli viene passata una **strategia**, estratta casualmente, che poi stabilisce anche la velocità massima a cui può andare il giocatore controllato dal bot.

In questo modo, ogni FormulaUnoBotPlayer viene controllato da un'istanza di ControllerFormulaUnoBotSimulator, che agisce seguendo la velocità massima, dettata di volta in volta dalla strategia del singolo player.
Si possono avere 5 strategie diverse, rappresentate dalle enumerazioni:

- *WISETURTLE* - velocità massima: 1
- *SPEEDYTURTLE* - velocità massima: 2
- *OOGWAY* - velocità massima: 3
- *RABBIT* - velocità massima: 4
- *BEEPBEEP* - velocità massima: 5

Le strategie, assegnate casualmente, fanno in modo che la simulazione sia diversa ogni volta che si avvia l'esecuzione, anche se può capitare di ottenere le stesse combinazioni di giocatori e le rispettive strategie, ma è solo un fattore di probabilità.

Questa soluzione permette di far sfidare fra di loro giocatori "più o meno temerari", a seconda della velocità massima a cui corrono!

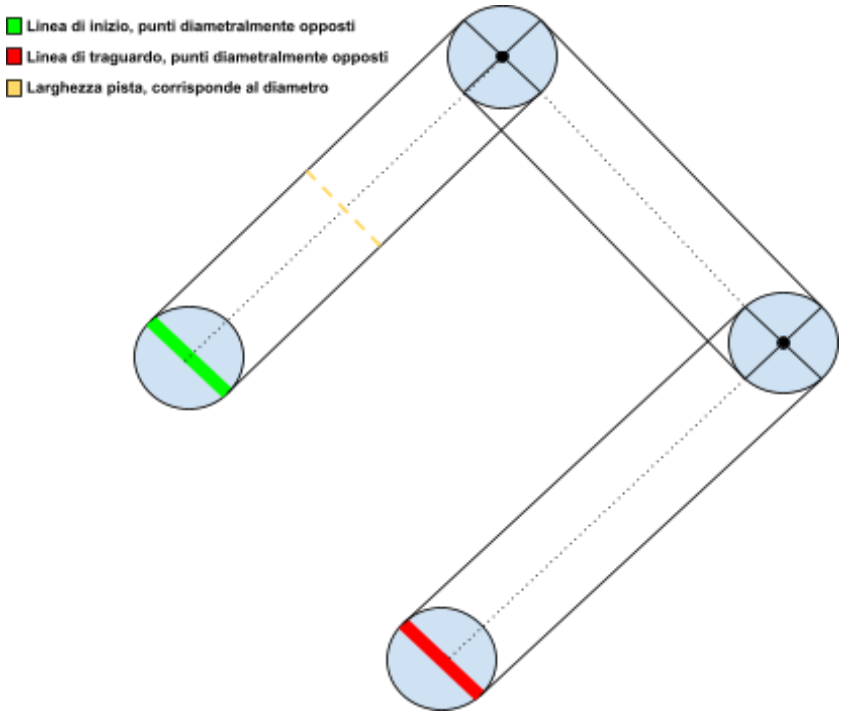
Metodo `getMove()`:
Funzionalità: Determina la prossima mossa del giocatore bot.
Se non ci sono mosse legali, gestisce una collisione o la fu

Altrimenti, sceglie la mossa migliore e verifica se la velocità è troppo alta.
Se il bot ha raggiunto la linea del traguardo, gestisce la vittoria.
Verifica se il bot è uscito dalla pista o ha avuto una collisione.

Metodo `chooseBestMove()`:
Funzionalità: Sceglie la mossa migliore basata sulla distanza dalla fine della sezione corrente e dalla posizione attuale.

Struttura e logica della pista

La pista è composta da delle sezioni, che hanno un punto di inizio e un punto di fine.
Ogni punto è definito dalle coordinate, successive alla prima riga, passate nel file di testo, la prima riga, invece, indica il raggio del “cerchio” che si va a creare attorno ad ogni punto. Questo raggio va a definire di conseguenza la larghezza della pista, poiché per creare le sezioni si sfruttano i punti diametralmente opposti nel cerchio in creando una linea retta perpendicolare alla coordinata successiva.
Ecco un esempio visivo:



Questa logica di creazione del tracciato consente di realizzare piste che hanno anche tratti sovrapposti, visto che la creazione delle sezioni avviene in modo sequenziale e si evitano interferenze fra i dati.

Per evitare che le mosse dei giocatori controllati dal bot vadano fuori pista, sono stati creati metodi che verificano, suddividendo algoritmicamente la traiettoria della mossa,

se esse vanno fuori pista con il metodo getIntegerRoutePoints nella classe *Coordinate* in concomitanza del metodo IsIntoMap nella classe *FormulaUnoMap*, sfruttando il metodo IsIntoSection della classe *FormulaUnoSection*.

La vittoria si verifica quando il giocatore arriva nella sezione di arrivo.

Avvio dell'applicazione

Per avviare l'applicazione, eseguire i seguenti comandi:

Compilazione: `gradle clean build`
Esecuzione 1: `gradle run`, che fa partire l'applicativo con una mappa un numero di giocatori di default.

Esecuzione 2: Accedere alla directory `build/libs` e avviare il jar con il comando `java -jar <file-jar>` seguito dagli argomenti necessari per la view:

`-mp <path>`: Specificare il percorso del file contenente i dati custom del tracciato. (il path è relativo alla posizione del .jar)

`-pn <number>`: Specificare il numero dei giocatori.

Simulazione

Per eseguire una simulazione, nella cartella del progetto è disponibile una cartella "**dist**" in cui è presente il jar pre compilato con una mappa prefatta.
L'unico dato mancante è il numero di giocatori che si assegna su linea di comando manualmente (sostituire <number>).

Quindi aprire la cartella nel terminale ed eseguire i comandi:

```
java -jar .\ProgettoF1.jar -mp .\map1.txt -pn <number>
```

Esempio di file custom (map1.txt):

```
5 //raggio del cerchio, quindi 10 è la larghezza della pista
50,70
80,40
100,60
-45,79
300,25
```

La prima riga indica la **larghezza** della pista, e le righe successive definiscono le **coordinate** del tracciato.

Conclusione

L'applicativo di simulazione del gioco Pencil Racing offre una piattaforma flessibile e personalizzabile per simulare gare. Con una struttura modulare e una gestione attenta delle mosse e delle condizioni di gioco, rappresenta un esempio pratico di applicazione di simulazione basata su console. La possibilità di estendere il sistema per supportare giocatori umani e di personalizzare la pista rende l'applicativo versatile e adattabile a diverse esigenze.